

MOSS- Mini Operating System Services Simulator

By: Fletcher Baccus, Jaden Leonard,
Amir Taylor, Bruce Uzoma

```
(fletcher@kali)-[~/csc414-514-final-project]
$ ./moss
MOSS - Mini Operating System Services Simulator
Type 'help' for commands

moss> help
MOSS - Mini Operating System Services Simulator

Subsystem A - Scheduling:
  sched create_process <arrival> <burst> <priority>
  sched schedule <FCFS|RR|PRIORITY> [quantum]
  sched run | gantt | stats | reset

Subsystem B - Memory:
  mem algorithm <FIFO|LRU>
  mem access <address> | frames | stats | reset | demo

Subsystem C - Sync:
  sync read | write <USER|EDITOR|ADMIN>
  sync check <USER|EDITOR|ADMIN> <READ|WRITE|MANAGE>
  sync permissions | status | reset | demo

Type 'sched help', 'mem help', or 'sync help' for detailed usage.
demo    run vertical slice (end-to-end integration demo)
exit    quit
```

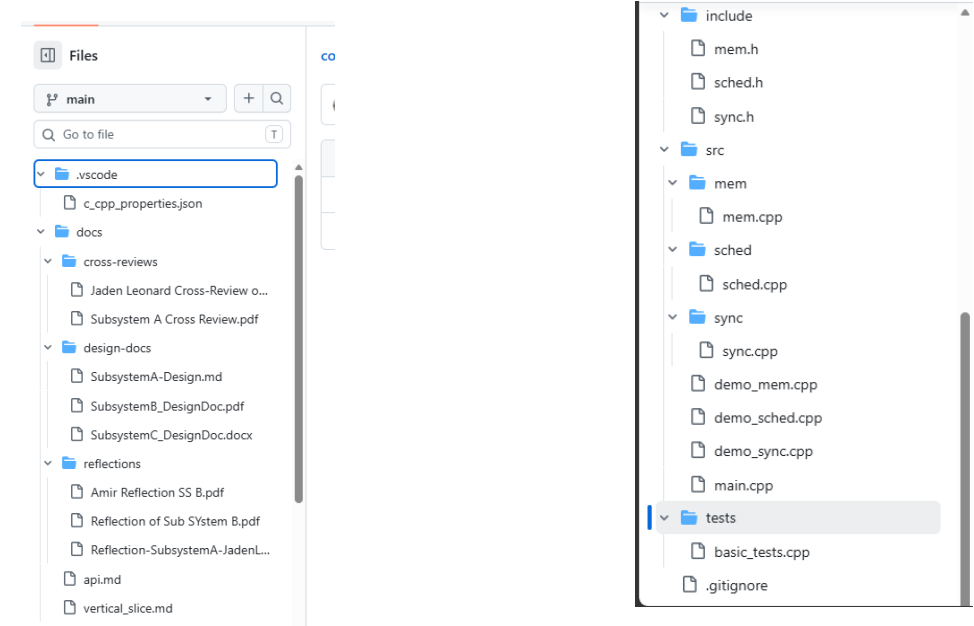


Project Overview

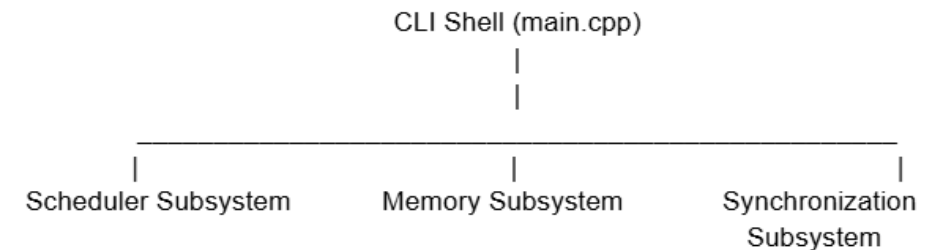
- **Objectives:**
 - Develop a operating system simulator
 - Model core OS services and subsystem interaction
 - Demonstrate scheduling, memory management, and synchronization
- **Motivation For Project:**
 - Real operating systems use subsystem coordination
 - Modular development improves maintainability and scalability
 - Simulators help visualize complex OS concepts

System Architecture

- **Core Components**
- Command-line interface (CLI)
- Scheduling subsystem
- Memory management subsystem
- Synchronization and protection subsystem
- **Design Approach**
- Modular API-driven structure
- Independent subsystem implementation
- Unified integration through main command shell



System Architecture



Subsystem A-Process Scheduling

- **Features**
- Process creation and management
- 3 Scheduling Algorithms
- Gantt chart generation
- Waiting and turnaround time calculation
- **Algorithms Implemented**
- First Come First Serve (FCFS)
- Round Robin (RR)
- Priority Scheduling (Non-Preemptive)
- **Purpose**
- Simulate CPU process scheduling behavior
- Compare execution fairness and efficiency

```
moss> sched create_process 0 5 1
      created process P0
moss> sched create_process 1 3 2
      created process P1
moss> sched schedule FCFS
      algorithm set to FCFS
moss> sched run
      simulation complete
```

PID	Wait	Turnaround
0	0	5
1	4	7

Avg wait: 2.00 Avg turnaround: 6.00

```
moss> 
```

Subsystem B- Memory Management

- **Features**
- Paging simulation
- Logical-to-physical address translation
- Page hit and page fault tracking
- Frame allocation monitoring
- Replacement policy simulation
- **Algorithms Implemented**
- FIFO Page Replacement
- LRU Page Replacement
- **Purpose**
- Simulate virtual memory behavior
- Demonstrate memory access efficiency

```
moos> mem demo
Demo sequence: 1 2 3 4 1 2 5 1 2 3 4 5
FAULT page=1 addr=0x1000 phys=0x0
Frames: [1 -- -- ]
FAULT page=2 addr=0x2000 phys=0x100
Frames: [1 2 -- ]
FAULT page=3 addr=0x3000 phys=0x200
Frames: [1 2 3 -- ]
FAULT page=4 addr=0x4000 phys=0x300
Frames: [1 2 3 4 ]
HIT page=1 addr=0x1000 phys=0x0
Frames: [1 2 3 4 ]
HIT page=2 addr=0x2000 phys=0x100
Frames: [1 2 3 4 ]
FAULT page=5 addr=0x5000 phys=0x0
Frames: [5 2 3 4 ]
FAULT page=1 addr=0x1000 phys=0x100
Frames: [5 1 3 4 ]
FAULT page=2 addr=0x2000 phys=0x200
Frames: [5 1 2 4 ]
FAULT page=3 addr=0x3000 phys=0x300
Frames: [5 1 2 3 ]
FAULT page=4 addr=0x4000 phys=0x0
Frames: [4 1 2 3 ]
FAULT page=5 addr=0x5000 phys=0x100
Frames: [4 5 2 3 ]
Hits: 2 Faults: 10 Hit rate: 16.7%
moos>
```

Logical Address -> Page Table -> Physical Frame

```

mosss> sync permissions

```

Role	READ	WRITE	MANAGE
USER	YES	no	no
EDITOR	YES	YES	no
ADMIN	YES	YES	YES

```

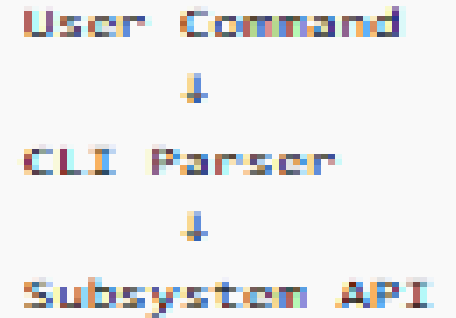
mosss>

```

Subsystem C- Synchronization and Protection

- **Features**
- Role-based permission system
- Read/write synchronization
- Protection checks
- Shared resource access control
- **Roles Implemented**
- USER
- EDITOR
- ADMIN
- **Purpose**
- Simulate synchronization and system protection
- Prevent unauthorized operations

Subsystem Integration



- **Integration Goals**

- Combine independent subsystems into one simulator
- Maintain modularity and subsystem separation
- Enable unified command execution

- **Integration Techniques**

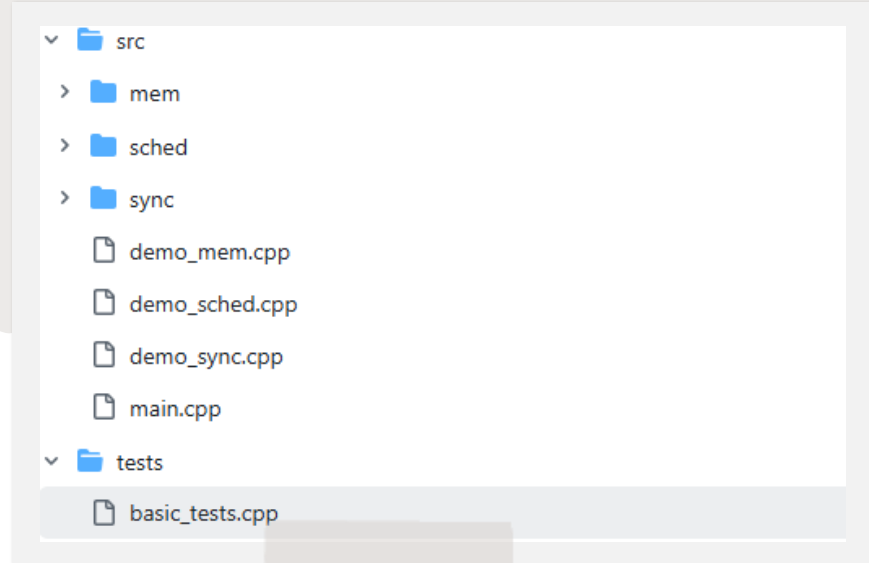
- Shared APIs
- Unified command parser
- Makefile-based compilation
- Centralized CLI shell

- **Key Integration Component**

- main.cpp routes user commands to subsystems

```
if (token == "exit" || token == "quit") break;
else if (token == "help") print_help();
else if (token == "demo") run_vertical_slice();
else if (token == "sched") handle_sched(ss);
else if (token == "mem") handle_mem(ss);
else if (token == "sync") handle_sync(ss);
else std::cout << " unknown command: " << token << " (type 'help')\n";
```

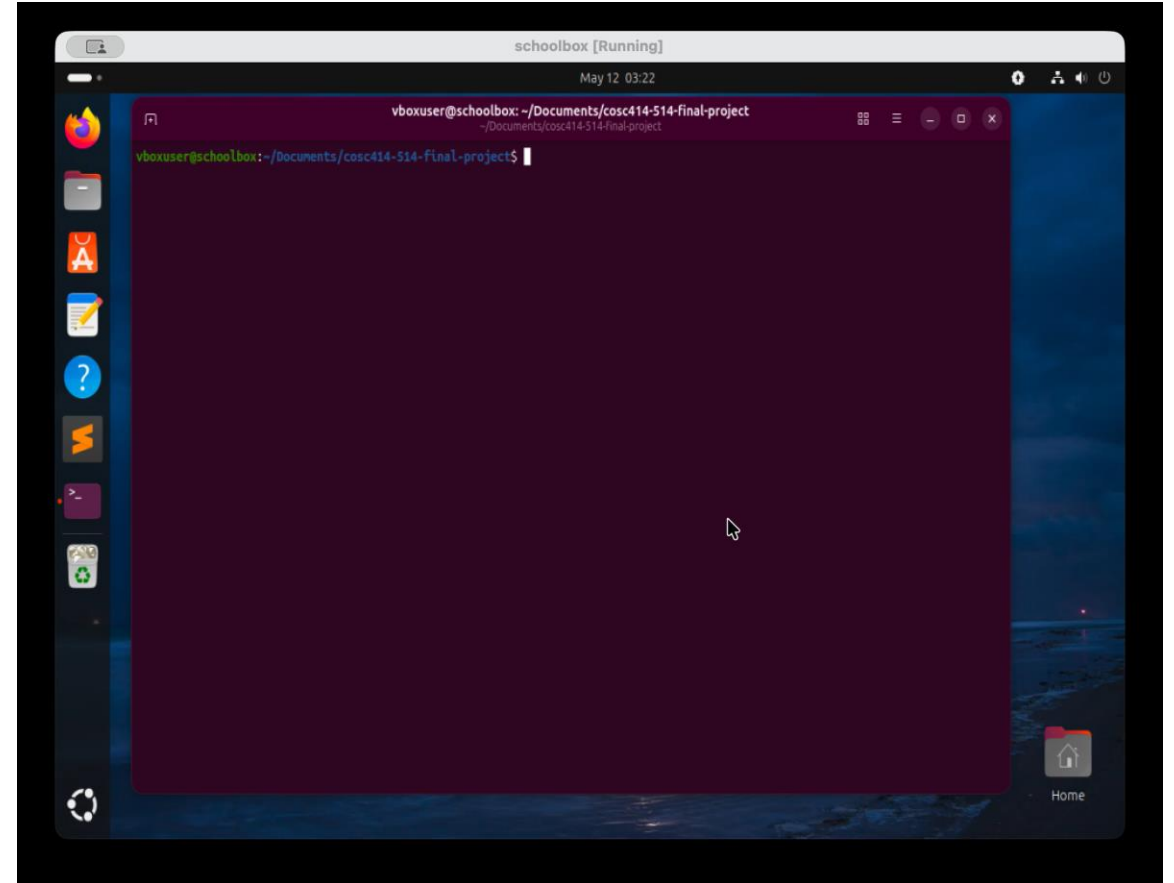
Integration Challenges and Solutions



- **Challenges**
 - Combining the modules together
 - Maintaining consistent interfaces
 - Debugging subsystem interactions
 - Synchronizing subsystem communication
- **Solutions**
 - Modular API design
 - Shared interface specifications
 - Testing and integration
 - Centralized build management using Make

Vertical Slice Demonstration

- **End-to-End Workflow**
- Process creation
- CPU scheduling execution
- Memory access handling
- Synchronization and protection checks
- **Demonstration Goals**
- Show subsystem interaction
- Validate integrated system behavior
- Demonstrate complete execution flow



Results and Analysis

- **Successful Outcomes**
 - Correct process scheduling behavior
 - Accurate page hit and fault tracking
 - Functional permission enforcement
 - Successful integration between subsystems
- **Performance Metrics**
 - Waiting time calculations
 - Turnaround time calculations
 - Memory hit rates
 - Scheduling execution order
- **Observations**
 - Round Robin improved scheduling fairness
 - Memory hit rate increased with repeated accesses
 - Permission system correctly denied restricted action

KALI [Running] - Oracle VirtualBox

File Machine View Input Devices Help

1234

fletcher@Kali: ~/cosc414-514-final-project

File Actions Edit View Help

Hits: 4 Faults: 8 Hit rate: 33.3%

moss> mem reset

frames and stats cleared

moss> mem algorithm FIFO

algorithm set to FIFO

moss> mem access 256

FAULT page=1 offset=0 phys=0x0

Frames: [1 -- --]

moss> mem access 512

FAULT page=2 offset=0 phys=0x100

Frames: [1 2 --]

moss> mem access 768

FAULT page=3 offset=0 phys=0x200

Frames: [1 2 3 --]

moss> mem access 1024

FAULT page=4 offset=0 phys=0x300

Frames: [1 2 3 4]

moss> mem access 1280

FAULT page=5 offset=0 phys=0x0

Frames: [5 2 3 4]

moss> quit

(fletcher@Kali)-[~/cosc414-514-final-project]

\$

When summary appears:

Say:

The statistics show the total number of hits, faults, and overall hit rate for the FIFO algorithm.

NOW SHOW LRU

THIS is the important part you were missing.

Type:

Enter command

Say:

I am resetting memory and statistics before testing LRU.

Now I am switching to the least recently used replacement algorithm.

Then:

Enter command

4. Appending...

46°

1:33 AM



Limitations & Future Improvements

- **Limitations**
 - Command-line only interface
 - Limited scheduling policies
 - Single-system simulation environment
- **Future Improvements**
 - Graphical user interface (GUI)
 - Advanced memory allocation strategies
 - Deadlock detection and prevention
 - Dynamic process generation
 - Real-time scheduling algorithms

Questions?